https://doi.org/10.1007/sxxxxx-yyy-zzzz-1

RESEARCH ARTICLE

VulTriage: Auditable Automation under Project Shift for Vulnerability Detection

Yanbo Bian[1]✉, Zhihai Wang[2]✉

1. Computer Science and Technology Program, Weihai International College, Beijing Jiaotong University, Weihai 264401, China
2. School of Computer Science and Technology, Beijing Jiaotong University, Beijing 100044, China

Received month dd, yyyy; accepted month dd, yyyy
E-mail: 24722081@bjtu.edu.cn; zhhwang@bjtu.edu.cn.

Abstract

Vulnerability detectors moved to an unseen repository face asymmetric costs: excluding the vulnerable label can miss a defect, whereas excluding the safe label can create an avoidable alert. A confidence score alone does not reveal whether the labeled source evidence supports a requested pair of class-specific risk budgets on the new project. We introduce VulTriage, a deployment layer that combines class-conditional set-valued decisions, cross-fitted source–target density-ratio estimates, and a fail-closed support audit. It returns a singleton label only at support-qualified operating points and otherwise sends the function to review. We evaluate the protocol on the public PrimeVul release using 12 frozen project-disjoint targets, five technical seeds, nine preregistered budget pairs, 17 method variants, and prediction artifacts sealed before target-label access. At a 10% vulnerable-class budget, 10 of 12 projects pass the label-free support audit; at the paired 20% safe-class budget, these projects retain 57.83% median singleton coverage and issue singleton decisions for both true classes. Across the full grid, estimated weighting reduces the maximum class-specific budget violation in 74 of 108 project–operating-point pairs. Support qualification is not a target-risk certificate: after label release, 6 of the 10 supported projects meet the vulnerable budget, 5 meet the safe budget, and 1 meets both after seed averaging. The positive result is therefore an auditable partial-automation protocol with explicit refusal and measurable coverage, not a guarantee under arbitrary project shift.

Keywords

Vulnerability detection; Conformal prediction; Selective prediction; Project shift; Uncertainty; Software security

1 Introduction

A vulnerability detector deployed on a new software project must do more than rank functions. It must decide whether an output is defensible enough to automate. This distinction matters because a false "safe" decision may hide a vulnerable function, whereas a false vulnerability flag consumes scarce review effort. The two errors are not interchangeable, and their acceptable rates depend on the deployment context.

Recent datasets and audits have made vulnerability evaluation more realistic. PrimeVul exposes the effects of duplicates, label quality, natural class imbalance, and evaluation design on code-language-model results [1]; DiverseVul similarly documents weak generalization across diverse projects and weakness categories [2]. Cross-project detectors address representation or training mismatch through transfer learning and domain adaptation [3–6]. These methods improve the predictor, but deployment still requires a rule for deciding when its output should be automated.

Probability calibration and rejection provide part of that rule. Temperature scaling is a canonical probability-calibration baseline [7], and selective prediction explicitly trades risk against coverage [8]. Code-model calibration can nevertheless degrade under distribution shift [9], while program shifts affect uncertainty estimators differently [10]. Prom is the closest deployment-time conformal framework for code-analysis models and includes vulnerability detection [11]; recent preprint work also treats correctness estimation and deferral as a general code-prediction problem [12]. What remains missing is a security-specific interface that accepts separate vulnerable- and safe-class budgets, tests whether unlabeled target evidence supports the requested operating point, and exposes refusal rather than silently forcing a label.

VulTriage addresses that gap. It turns a binary detector into three actions: automatically return {safe}, automatically return {vulnerable}, or return {safe, vulnerable} for review. Class-conditional conformal scores encode asymmetric budgets. A cross-fitted domain classifier estimates target relevance, while effective-sample-size (ESS), neighborhood, and test-point-mass checks decide whether the operating point is support-qualified. An unsupported target fails closed to review. The use of estimated weights is empirical: exact weighted-conformal results require explicit covariate-shift and likelihood-ratio assumptions [13], and related conformal-risk frameworks retain their own exchangeability or nonexchangeability conditions [14–16].

We ask three research questions:

- **RQ1:** Where does support-aware triage enable nontrivial automation on unseen projects?
- **RQ2:** Does estimated target weighting improve alignment with the requested class-specific budgets?
- **RQ3:** What automation cost accompanies a fail-closed deployment rule?

This paper contributes:

1. an executable asymmetric triage protocol that places a label-free support audit before automatic decisions;
2. a positive, bounded deployment result: at $(\alpha_v, \alpha_s) = (0.10, 0.20)$, 10/12 unseen projects are support-qualified and retain 57.83% median singleton coverage;
3. a sealed, preregistered PrimeVul study showing that estimated weighting reduces worst-class budget violation in 74/108 project–operating-point pairs, together with the coverage and target-risk limitations of that result.

The evidence comes from one low-cost hashing/linear detector. We consequently claim neither state-of-the-art detection nor validation across detector families.

2 Background and Related Work

2.1 Cross-project vulnerability detection

Early cross-project vulnerability work transferred learned program representations between projects [17]. CD-VulD subsequently combined token and deep representations with metric transfer [18]. CPVD uses graph attention and domain adaptation [3], DAM2P combines discrepancy reduction and max-margin learning for imbalanced cross-project data [4], and CSVD-TF applies TrAdaBoost to expert and semantic features [5]. ZSVulD studies a zero-shot setting with pretrained embeddings, pseudo-labeling, and collaborative classifiers [6]. These studies establish that cross-project prediction and unlabeled-target adaptation are occupied research areas. VulTriage is complementary: it does not propose a new program representation; it decides whether a post-hoc set-valued action is support-qualified.

Modern detector research also improves semantic modeling. CodeBERT provides a widely used pretrained code representation [19]; LineVul adapts transformer representations to function prediction and line localization [20]; DeepDFA injects data-flow structure for efficient detection [21]. Because our completed experiment uses a hashing/SGD detector, these works define an important external-validity boundary rather than numerically commensurate baselines.

2.2 Calibration, selection, and code-model deferral

Calibration, ranking, and selective action answer different questions. Temperature scaling changes probability calibration but not the ordering of binary confidence scores [7]. SelectiveNet represents the learned-rejection family and motivates reporting risk together with retained coverage [8]. For code models, Zhou et al. report overconfidence and weaker calibration remedies under out-of-distribution shifts [9], and Li et al. show that temporal, project, and author shifts affect uncertainty methods differently [10].

Prom uses multiple conformal experts—LAC, TopK, APS, and RAPS—with local weighting and voting to assess deployment-time reliability across code-analysis tasks [11]. Our experiment implements only a global label-conditional LAC-credibility variant at matched singleton counts; we label it *Prom-compatible*, not the official Prom ensemble. VulTriage differs in its separate security budgets, frozen project-disjoint PrimeVul protocol, and explicit whole-target support refusal. Generic code deferral is also not new [12]; the contribution here is the asymmetric support-gated deployment protocol and its evaluation.

2.3 Conformal prediction under shift

Weighted conformal prediction can recover target marginal coverage under covariate shift when $P(Y \mid X)$ is invariant and the target-to-source likelihood ratio is known [13]. Conformal risk control extends calibration to bounded monotone losses [14]; nonexchangeable conformal risk control incorporates relevance weights and an explicit loosening term [15]. High-probability risk control under covariate shift provides another importance-weighted formulation for label-dependent risks [16]. We inherit the test-point-aware weighted quantile but not an oracle guarantee: our ratios come from a learned domain classifier, so target results are reported empirically and the support monitor is an operational safeguard.

3 VulTriage

3.1 Three-way decision problem

Let $Y \in \{0, 1\}$ denote safe and vulnerable, respectively, and let a fitted detector return $\widehat{p}_y(x)$ for each class. For a calibration pair (x_i, y_i), the label-conditional nonconformity score is

$$S_i = 1 - \widehat{p}_{y_i}(x_i). \tag{1}$$

For a test function x, VulTriage returns a set $\Gamma(x) \subseteq \{0, 1\}$. A singleton is an automatic decision; the doubleton is manual review. Empty sets produced by comparison methods are also counted as review and reported separately.

For class y, miscoverage is

$$R_y = \Pr\{y \notin \Gamma(X) \mid Y = y\}. \tag{2}$$

The deployment interface specifies α_v for the vulnerable class and α_s for the safe class. Singleton coverage is $\Pr\{|\Gamma(X)| = 1\}$; review load is its complement after including empty and doubleton outputs.

3.2 Class-asymmetric weighted calibration

For class y, let $\mathcal{I}_y$ index calibration examples and let $w_i \geq 0$ be their estimated target relevance. For test point x with weight $w(x)$, the implemented threshold is

$$q_y(x) = \inf\left\{ t : \sum_{i \in \mathcal{I}_y} w_i \mathbf{1}(S_i \leq t) \geq (1 - \alpha_y)\left(\sum_{i \in \mathcal{I}_y} w_i + w(x) \right) \right\}. \tag{3}$$

If the calibration mass cannot reach the right-hand side, $q_y(x) = \infty$, which is the test-point mass-at-infinity construction. The prediction set is

$$\Gamma(x) = \{y : 1 - \widehat{p}_y(x) \leq q_y(x)\}. \tag{4}$$

Unweighted Mondrian conformal prediction is recovered by unit calibration weights with the corresponding finite-sample higher quantile.

3.3 Cross-fitted project-shift weights

For each held-out project, a domain classifier distinguishes source-calibration examples ($D = 0$) from unlabeled target examples ($D = 1$). Training folds contain equal source and target counts; therefore the out-of-fold target odds provide

$$\widehat{w}(x) = \frac{\widehat{P}(D = 1 \mid x)}{1 - \widehat{P}(D = 1 \mid x)}. \tag{5}$$

Row hashes deterministically assign five cross-fitting folds, so no example receives an in-sample domain probability. Ratios are clipped to $[1/20, 20]$ for the primary method; clips 10 and 50 are sensitivity variants. The same label-free ratio estimate is shared across detector-head seeds.

3.4 Fail-closed support gate

The gate evaluates four frozen conditions before labels are released. For weights $w_1, \ldots, w_m$, Kish ESS is

$$\mathrm{ESS}(w) = \frac{(\sum_i w_i)^2}{\sum_i w_i^2}. \tag{6}$$

Total ESS must be at least 200. For each class y, ESS must be at least $\max\{20, \lceil 2/\alpha_y \rceil\}$, and at least 10 calibration examples must retain positive weight. Finally, every target point must have test-point infinity mass no larger than both requested class budgets. If any condition fails, or if the resulting target contains no singleton, the entire project–budget output is replaced by doubletons. Figure 1 shows that this audit precedes automation.

4 Preregistered Study

4.1 Data and frozen project splits

We use the original public PrimeVul release [1]. Our audit reproduces 235,768 unique functions: 6,968 vulnerable and 228,800 safe. The official chronological split contains 184,427 training, 25,430 validation, and 25,911 test examples, but it is not project-disjoint. We therefore report it only as a temporal track.

The primary track holds out each complete normalized project group in Table 1. Eligibility was frozen at at least 200 total, 20 vulnerable, and 20 safe functions. The other projects form the source pool. Source functions are partitioned by salted commit hash into 70% detector training, 10% model validation, and 20% calibration. Target labels are stored separately and unavailable to the prediction runner.

4.2 Detector and comparison methods

The completed base detector is deliberately low-cost. A label-free hashing vectorizer maps case-sensitive word unigrams and bigrams to $2^{18} = 262{,}144$ dimensions with ℓ_2 normalization. A class-balanced averaged SGD logistic head trains for three epochs. Its regularization $\{10^{-6}, 10^{-5}, 10^{-4}\}$ is selected by source-validation area under the precision–recall curve (PR-AUC), then the head is refit on source training plus validation. Five seeds, 13, 37, 73, 101, and 137, are technical repetitions.

Table 2 groups the 17 recorded method labels. Confidence methods are matched to the requested conformal singleton count so that ranking quality is compared at the same automation volume. In binary classification, predictive entropy and maximum softmax probability induce the same ordering, which we mark explicitly. The Prom-compatible method uses global label-conditional LAC credibility, not the official four-expert Prom ensemble.

4.3 Outcomes, sealing, and inference

The frozen grid is $\alpha_v \in \{0.01, 0.05, 0.10\}$ by $\alpha_s \in \{0.05, 0.10, 0.20\}$. Primary outcomes are class-specific miscoverage, positive violation $\max(0, R_y - \alpha_y)$, singleton coverage, and review load. We also record PR-AUC, precision, recall, F-scores, Matthews correlation coefficient, Brier score, expected calibration error, and area under the risk–coverage curve.

Prediction generation produced 65 detector runs (the official track plus 12 project targets, each with five seeds) and 130 prediction/metadata files. Every file hash was sealed before evaluation labels were unlocked. Evaluation then generated 65 label-bearing decision archives and 9,495 unique metric rows. The metric-table SHA-256 is `1B6DFC08...1D27181`.

Seeds are averaged within a target project before inference; the independent unit is the 12 project groups. Paired median differences use 10,000 project-bootstrap resamples. Exact Wilcoxon tests are emitted only when at least 10 project differences are nonzero, and their p-values are Holm-adjusted within each outcome–budget family. We report descriptive counts when that gate is not met.

5 Results and Discussion

5.1 RQ1: support-qualified automation

Figure 2 identifies a broad positive operating region. No target passes every frozen support rule at $\alpha_v = 0.01$. Three targets pass at $\alpha_v = 0.05$, whereas 10 of 12 pass at $\alpha_v = 0.10$ for every safe-budget column. Thus the vulnerable budget, through its class-ESS and infinity-mass requirements, is the limiting axis of the support audit.

At $\alpha_v = 0.10$, increasing α_s from 0.05 to 0.10 and 0.20 raises median singleton coverage among the 10 supported projects from 38.64% to 44.83% and 57.83%, respectively. The 10%/20% point is therefore the clearest positive operating point in the preregistered grid: it support-qualifies 83.3% of target projects and automates a median 57.83% of functions within those targets.

Figure 3 shows that this automation is not concentrated in one repository. ImageMagick and GPAC fail closed, while the remaining ten projects have mean singleton coverage ranging from 45.93% (tcpdump) to 71.99% (OpenSSL). Every supported project produces nonzero singleton decisions among both truly vulnerable and truly safe functions.

Support qualification measures adequacy of the weighted calibration evidence, not held-out risk satisfaction. After labels are released and five seeds are averaged within project, 6/10 supported projects meet $R_1 \le 0.10$, 5/10 meet $R_0 \le 0.20$, and 1/10 meets both. At the seed–project level, 4/50 repetitions meet both. The positive conclusion is therefore the availability and breadth of audited partial automation, not a risk certificate.

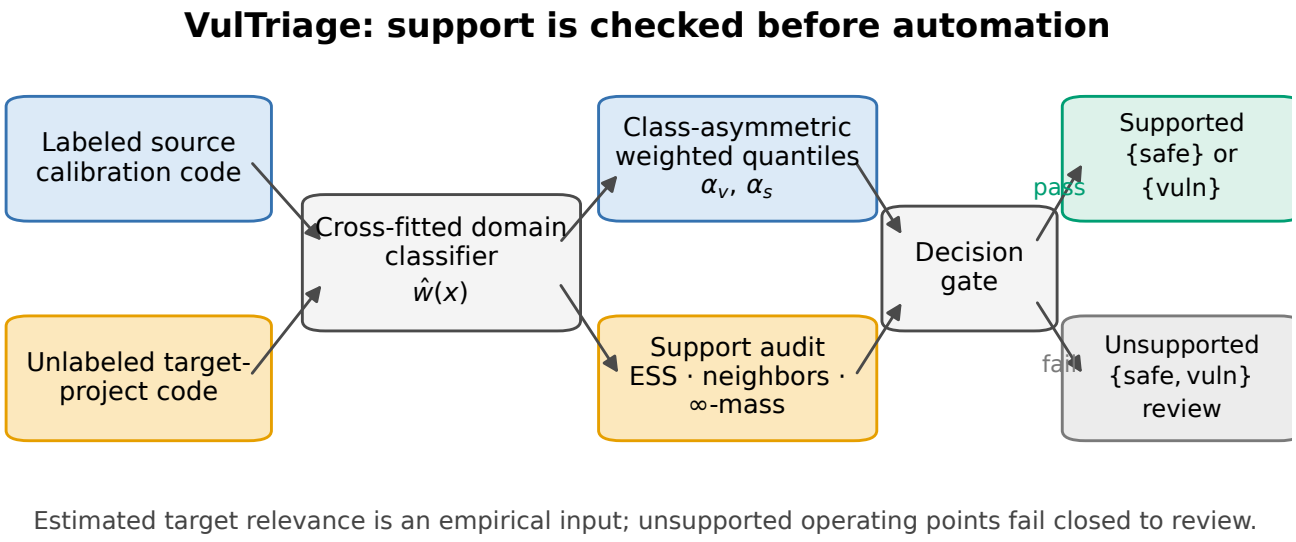

Fig. 1 VulTriage uses labeled source-calibration code and unlabeled target-project code to estimate relevance, construct class-asymmetric weighted thresholds, and audit support before automation. Estimated relevance is an empirical input; a failed gate returns the doubleton for review rather than a forced label.

Table 1 Frozen project-disjoint PrimeVul target groups. Prevalence is vulnerable functions divided by total functions.

Project	Total	Vulnerable	Prev. (%)	Project	Total	Vulnerable	Prev. (%)
Linux	60,927	1,391	2.28	OpenSSL	2,151	156	7.25
Chrome	18,447	398	2.16	FFmpeg	2,834	121	4.27
ImageMagick	6,845	484	7.07	Vim	3,072	108	3.52
TensorFlow	3,058	247	8.08	tcpdump	648	101	15.59
PHP	6,261	269	4.30	GPAC	5,922	89	1.50
QEMU	6,328	177	2.80	radare2	2,860	77	2.69

Table 2 Comparison families and their role in the study.

Family	Variants
Forced	Argmax label for every function
Confidence	MSP, binary-equivalent entropy, and temperature-scaled MSP at matched singleton counts
Unweighted conformal	Mondrian and pooled split conformal
Code deployment	Global label-conditional LAC credibility at matched counts; not official Prom
Shift weighting	Test-point-aware estimated weights with clips 10, 20, and 50
Fail-closed	VulTriage with the same three clips plus matched selection controls

5.2 RQ2: directional risk alignment

To isolate weighting from fail-closed replacement, Fig. 4 compares raw estimated-weight prediction sets with unweighted Mondrian sets. For each of 12 projects and nine budget pairs, we average over seeds and take the larger vulnerable- or safe-class violation. Estimated weighting decreases this worst-class violation in 74/108 pairs (68.5%), ties in 7, and increases it in 27. Improvements occur throughout the grid: 24/36, 25/36, and 25/36 pairs at vulnerable budgets 1%, 5%, and 10%, respectively.

The improvement is not free utility. Raw weighting increases singleton coverage in only 2/108 pairs. This trade-off is reported beside the support region and fail-closed cost so that risk alignment is not mistaken for general method dominance.

5.3 RQ3: cost of failing closed

The preregistered 5%/10% point illustrates the trade-off. Versus unweighted Mondrian prediction, fail-closed VulTriage lowers vulnerable-class violation in seven projects and ties in five; it lowers safe-class violation in six and ties in six. These outcomes do not meet the frozen Wilcoxon gate because fewer than ten project differences are nonzero.

The coverage cost is large and statistically clear. Across all 12 targets, VulTriage reduces singleton coverage by a median 37.72 percentage points relative to unweighted Mondrian (95% project-bootstrap interval $[-44.84, -24.72]$; Holm-adjusted exact Wilcoxon $p = 0.001953$). Only OpenSSL, PHP, and QEMU are support-qualified at this point, although all three retain singleton decisions for both true classes and have 31.37% median singleton coverage. Fail-closed review is thus the price paid for explicit support refusal, not a hidden accuracy gain.

5.4 Discussion

5.4.1 An operational deploy-or-refuse interface

The central positive result is a usable decision interface. A deployer chooses asymmetric budgets, fits the domain estimator from source

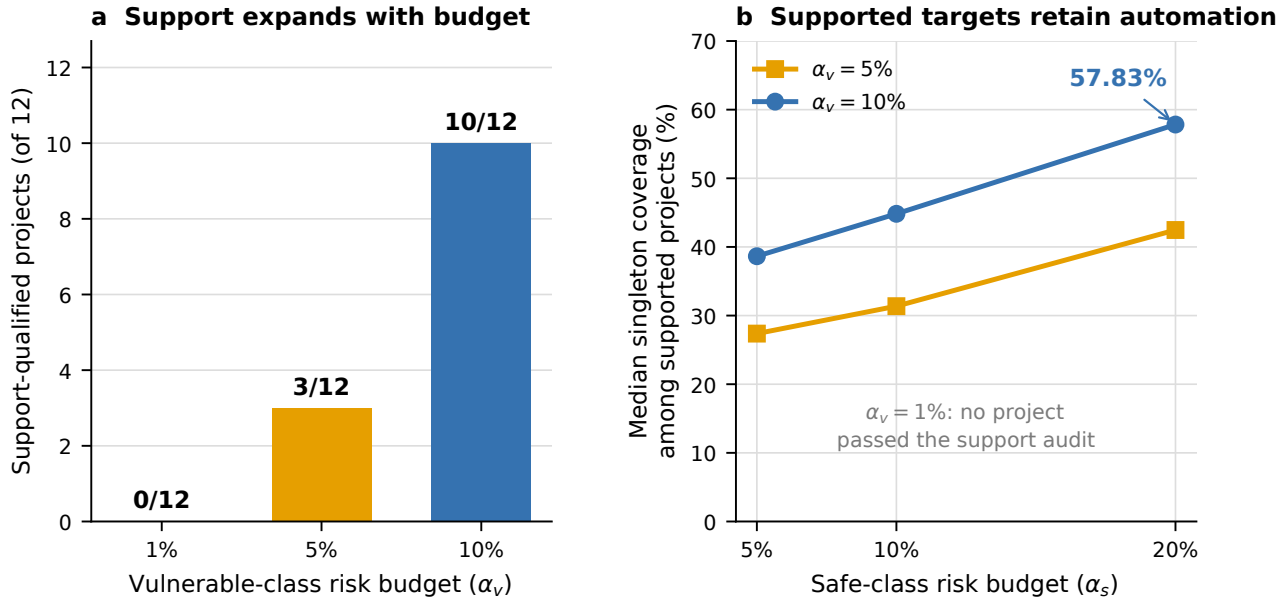

Fig. 2 Support-qualified automation over the preregistered budget grid. (a) The number of projects passing all frozen label-free support rules depends on the vulnerable-class budget and is unchanged across safe-budget columns. (b) Among supported projects, the safe-class budget controls retained singleton coverage. Coverage for the 1% vulnerable row is not plotted because no project was support-qualified; it is not treated as observed zero utility.

calibration and unlabeled target code, inspects the support decision, and obtains either set-valued automation or an explicit review-only result. At 10%/20%, this procedure clears 10/12 evaluated projects while retaining more than half of functions as singleton decisions at the median. The output therefore communicates both what the detector proposes and whether the calibration evidence is adequate for that operating point.

This interface improves on forced prediction in one important sense: it makes insufficient evidence visible. It does not establish that a cleared target will attain both requested risks. The empirical attainment counts show why the distinction matters. Deployment should log both the pre-label support diagnostics and any later audited outcomes; a gate can prevent unsupported automation without converting concept shift into covariate shift.

5.4.2 Why the support boundary appears

The observed $0/12 \to 3/12 \to 10/12$ pattern is consistent with the finite weighted evidence required by Eq. (3). The vulnerable class is rare, so $\lceil 2/\alpha_v \rceil$ and the test-point infinity mass become stringent as α_v decreases. At 1%, even substantial total calibration sets cannot supply adequate vulnerable-class weighted support for every target point. Relaxing the budget reduces the required effective evidence and makes partial automation available to more projects. This is a mechanistic interpretation of the frozen diagnostics, not a formal explanation of every target error.

Estimated weighting also changes the risk–coverage allocation. Its 74/108 directional wins show broad risk alignment, but the 2/108 coverage wins show that it usually removes rather than creates automatic decisions. This is appropriate for a gate whose purpose is defensibility, yet it limits claims of practical efficiency. A deployment with expensive review should treat the budget grid as a workload planning surface, not optimize risk alone.

5.4.3 Relationship to prior work

Domain-adaptive vulnerability detectors modify features, objectives, or pseudo-labels to improve prediction transfer [3, 4, 6]; VulTriage can sit after such a detector but does not replace it. Prom supplies a broader multi-expert deployment assessment for code tasks [11]; VulTriage narrows the interface to class-asymmetric vulnerability costs and makes unsupported-target handling explicit. Weighted conformal and conformal-risk-control papers provide the formal vocabulary [13–16]; our contribution is the empirical system and project-disjoint evidence, not a new theorem.

6 Limitations, Reproducibility, and Conclusion

6.1 Threats to validity

Construct validity. Singleton coverage is a proxy for automation volume, not analyst time saved. We did not conduct a human study and make no productivity claim. A {safe} singleton is a model decision, not security certification. Support qualification, empirical budget attainment, calibration error, and error ranking are reported as distinct constructs.

Internal validity. Target labels were isolated from prediction generation, but public benchmark labels can still be incomplete or noisy. Project aliases were normalized through a frozen manifest; residual repository provenance errors are possible. The density-ratio estimator shares one label-free cross-fit realization across head seeds, so seed variation measures the detector head rather than every pipeline component.

Statistical conclusion validity. The independent sample is 12 projects, not 60 seed runs. Bootstrap intervals and paired tests reflect

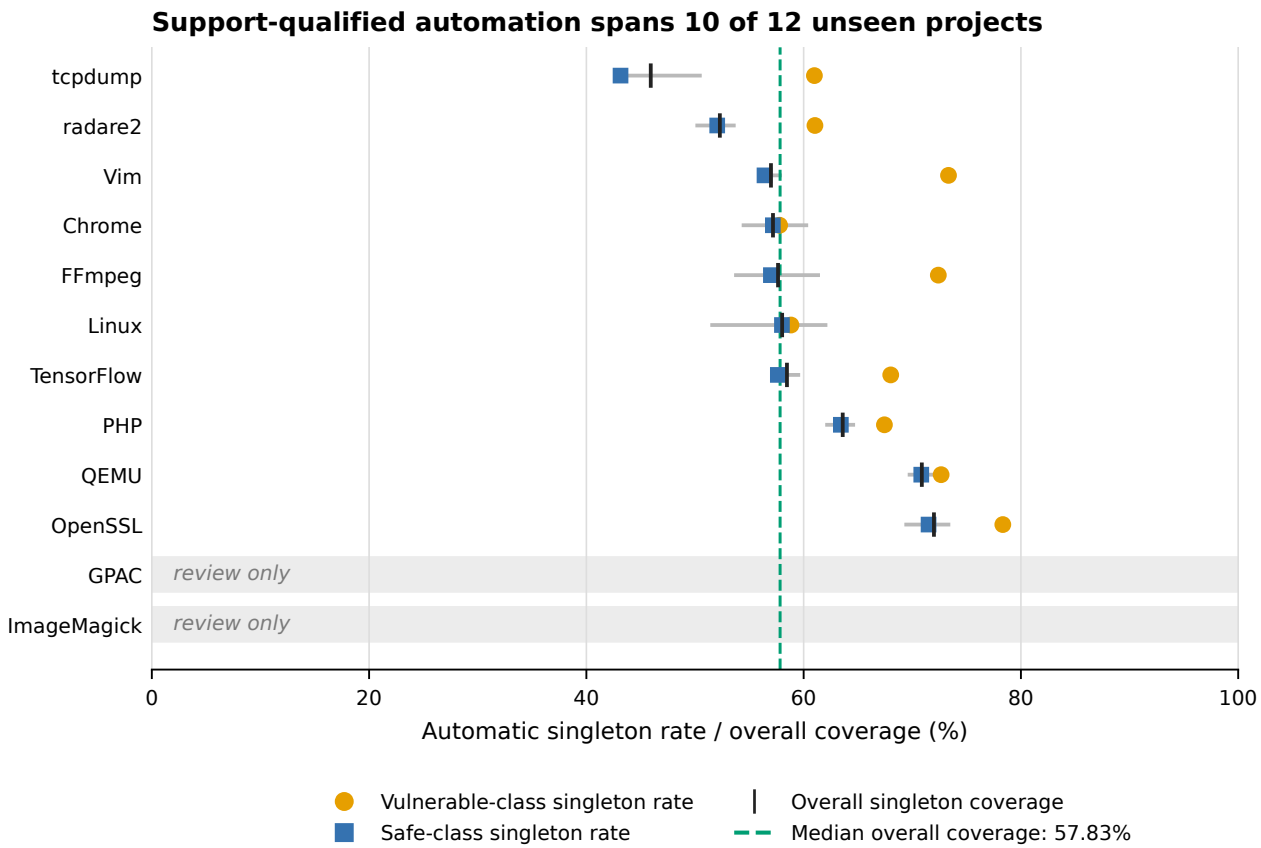

Fig. 3 Project-level automation at $(\alpha_v, \alpha_s) = (0.10, 0.20)$. Orange circles and blue squares are seed-averaged singleton rates within the vulnerable and safe true classes. Black ticks are overall singleton coverage; gray horizontal segments span the five technical seeds. The green dashed line is the 57.83% median overall coverage among supported projects. ImageMagick and GPAC fail the label-free support audit and are routed entirely to review.

project heterogeneity but remain imprecise at this size. We suppress Wilcoxon inference when fewer than ten nonzero project differences are present. The 74/108 grid count is descriptive and correlated within project.

External validity. The study uses one public C/C++ dataset and one hashing/SGD detector. On the official chronological track, forced classification has median PR-AUC 0.155 and misses most vulnerabilities at a 0.5% false-positive constraint (median FNR 0.917), confirming that the base detector is not state of the art. The positive result concerns the decision layer's auditable behavior, not detector quality or model-family invariance. Replication with CodeBERT-, LineVul-, and data-flow-based detectors [19–21] and a second dataset is required.

Theoretical boundary. Exact covariate-shift validity assumes invariant $P(Y \mid X)$ and known target/source ratios [13]. Our ratios are estimated from finite unlabeled target data, and project shift may include concept shift. ESS and infinity-mass checks are conservative diagnostics, not a proof of target coverage or risk control.

6.2 Reproducibility, data availability, and ethics

PrimeVul and its code are publicly available from the dataset authors' repository. The public VulTriage artifact contains the frozen configuration, split-manifest generator, label-free feature pipeline, prediction and evaluation code, statistical-analysis tables, plotting script, figure data, and immutable validation records. Key hashes are: configuration `9841A976...9500BE06`, split manifest `5AF519BF...16FE5EA`, prediction seal `DB564303...E0F8CF8`, and metric table `1B6DFC08...1D27181`. The exact E1 commands and complete hashes are included in the artifact. The repository is publicly available at `https://github.com/bianyanbo44-afk/vultriage-fcs`.

The study uses public source code and public vulnerability labels for defensive classification. It does not generate exploits, target live systems, or involve human participants. Institutional ethics review is therefore expected to be not applicable, subject to the authors' institutional policy.

6.3 Conclusion

VulTriage turns cross-project uncertainty into an explicit deploy-or-refuse decision. In a sealed, preregistered PrimeVul experiment, its label-free support audit admits 10 of 12 unseen projects at a 10% vulnerable-class budget; with a 20% safe-class budget, those projects retain 57.83% median singleton coverage. Estimated weighting improves worst-class budget alignment in 74 of 108 evaluated settings. The evidence supports a positive but bounded conclusion: auditable partial automation is attainable for most evaluated targets, while unsupported projects can be refused explicitly. It does not support universal target-risk guarantees, and stronger detectors and datasets remain essential

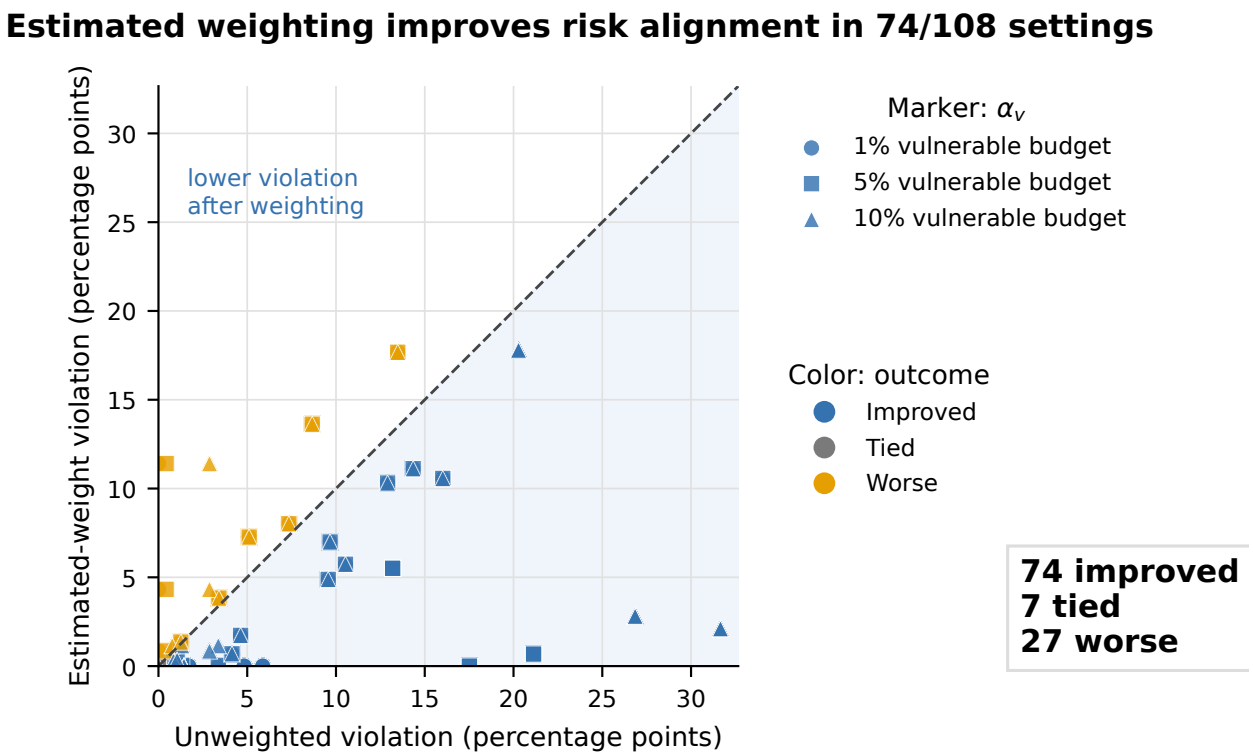

Fig. 4 Seed-averaged worst-class budget violation before and after estimated weighting. Points below the diagonal improve alignment; marker shape encodes the vulnerable-budget row and color encodes the outcome. Counts are descriptive over the frozen 12-project by nine-operating-point grid.

generalization tests.

Acknowledgements

The authors received no specific funding for this work.

Competing interests

The authors declare that they have no competing interests.

Author contributions

Yanbo Bian: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Project administration, Software, Validation, Visualization, Writing–original draft, Writing–review and editing. Zhihai Wang: Conceptualization, Formal analysis, Methodology, Supervision, Validation, Writing–review and editing. Both authors approved the final manuscript and agree to be accountable for the integrity of the work.

References

[1] Ding Y, Fu Y, Ibrahim O, Sitawarin C, Chen X, Alomair B, Wagner D, Ray B, Chen Y. Vulnerability detection with code language models: How far are we? In: 2025 IEEE/ACM 47th International Conference on Software Engineering. 2025, 1729–1741. doi: 10.1109/ICSE55347.2025.00038

[2] Chen Y, Ding Z, Alowain L, Chen X, Wagner D. DiverseVul: A new vulnerable source code dataset for deep learning based vulnerability detection. In: Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses. 2023, 654–668. doi: 10.1145/3607199.3607242

[3] Zhang C, Liu B, Xin Y, Yao L. CPVD: Cross project vulnerability detection based on graph attention network and domain adaptation. IEEE Transactions on Software Engineering, 2023, 49(8): 4152–4168. doi: 10.1109/TSE.2023.3285910

[4] Nguyen V, Le T, Tantithamthavorn C, Grundy J, Phung D. Deep domain adaptation with max-margin principle for cross-project imbalanced software vulnerability detection. ACM Transactions on Software Engineering and Methodology, 2024, 33(6): 1–34. doi: 10.1145/3664602

[5] Cai Z, Cai Y, Chen X, Lu G, Pei W, Zhao J. CSVD-TF: Cross-project software vulnerability detection with TrAdaBoost by fusing expert metrics and semantic metrics. Journal of Systems and Software, 2024, 213: 112038. doi: 10.1016/j.jss.2024.112038

[6] Haque R, Ali A, McClean S, Khan N. A zero-shot framework for cross-project vulnerability detection in source code. Empirical Software Engineering, 2026, 31(1): 3. doi: 10.1007/s10664-025-10749-4

[7] Guo C, Pleiss G, Sun Y, Weinberger K Q. On calibration of modern neural networks. In: Proceedings of the 34th International Conference on Machine Learning. 2017, 1321–1330. PMLR 70

[8] Geifman Y, El-Yaniv R. SelectiveNet: A deep neural network with an integrated reject option. In: Proceedings of the 36th International Conference on Machine Learning. 2019, 2151–2159. PMLR 97

[9] Zhou Z, Sha C, Peng X. On calibration of pre-trained code models. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024, 1–13. doi: 10.1145/3597503.3639126

[10] Li Y, Chen S, Yang W. Estimating predictive uncertainty under

program data distribution shift. arXiv preprint arXiv:2107.10989, 2021. arXiv:2107.10989

[11] Wang H, Lenihan P, Wang Z. Enhancing deployment-time predictive model robustness for code analysis and optimization. In: Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization. 2025, 31–46. doi: 10.1145/3696443.3708959

[12] Rathnasuriya R, Yang W. When to answer and when to defer: A decision framework for reliable code predictions. arXiv preprint arXiv:2605.19369, 2026. arXiv:2605.19369

[13] Tibshirani R J, Barber R F, Candès E J, Ramdas A. Conformal prediction under covariate shift. In: Advances in Neural Information Processing Systems 32. 2019. NeurIPS 2019

[14] Angelopoulos A N, Bates S, Fisch A, Lei L, Schuster T. Conformal risk control. In: International Conference on Learning Representations. 2024. arXiv:2208.02814; ICLR 2024

[15] Farinhas A, Zerva C, Ulmer D, Martins A F T. Non-exchangeable conformal risk control. In: International Conference on Learning Representations. 2024. ICLR 2024

[16] Almeida D C, Bravo J, Bono J, Bizarro P, Figueiredo M A T. High probability risk control under covariate shift. In: Proceedings of the Fourteenth Symposium on Conformal and Probabilistic Prediction with Applications. 2025, 133–152. PMLR 266

[17] Lin G, Zhang J, Luo W, Pan L, Xiang Y, De Vel O, Montague P. Cross-project transfer representation learning for vulnerable function discovery. IEEE Transactions on Industrial Informatics, 2018, 14(7): 3289–3297. doi: 10.1109/TII.2018.2821768

[18] Liu S, Lin G, Qu L, Zhang J, De Vel O, Montague P, Xiang Y. CD-VulD: Cross-domain vulnerability discovery based on deep domain adaptation. IEEE Transactions on Dependable and Secure Computing, 2022, 19(1): 438–451. doi: 10.1109/TDSC.2020.2984505

[19] Feng Z, Guo D, Tang D, Duan N, Feng X, Gong M, Shou L, Qin B, Liu T, Jiang D, Zhou M. CodeBERT: A pre-trained model for programming and natural languages. In: Findings of the Association for Computational Linguistics: EMNLP 2020. 2020, 1536–1547. doi: 10.18653/v1/2020.findings-emnlp.139

[20] Fu M, Tantithamthavorn C. LineVul: A transformer-based line-level vulnerability prediction. In: Proceedings of the 19th International Conference on Mining Software Repositories. 2022, 608–620. doi: 10.1145/3524842.3528452

[21] Steenhoek B, Gao H, Le W. Dataflow analysis-inspired deep learning for efficient vulnerability detection. In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. 2024, 1–13. doi: 10.1145/3597503.3623345